

Development of *Arnes Elektronik* (E-Commerce solution)

Building the Customer-Facing Frontend for *Arnes Elektronik* - A Multi-Team
Software Project

BSc in Software Engineering, Spring 2026

Semester: 2. semester

Course: Semesterprojekt: Development of Software Systems

Course-code: SI2-PRO

Project duration: February 2026 - May 2026

Authors:

Christian	—
Elias	—
Martin	—
Mikkel Blom	—
Yaroslav	—
Ziad	—

Group nr.: **T4G1**

Supervisor: Michael

Educational institution: Syddansk Universitet
Institute: Mærsk Mc-Kinney Møller Instituttet
Faculty: The Faculty of Engineering

Character count (incl. spaces): 68.685

Contents

1	Introduction	6
1.1	The Shop Module	6
2	Methodology	6
2.1	Agile Development Process	6
2.1.1	Scrum	7
2.1.2	Extreme Programming	7
2.2	Requirements	8
2.3	Team Organization	8
3	Scrum Reports	8
3.1	Product Backlog and Definition of Done	9
3.2	Sprint Summaries	9
4	Requirements	11
4.1	Requirements Elicitation	11
4.2	System Interfaces and Integration	12
4.3	Requirement Specification	12
4.4	Requirements Validation	14
4.5	Ongoing Management	14
5	Technical	14
5.1	System Modeling and UML	14
5.1.1	Class Diagram	14
5.1.2	Package Structure in the Class Diagram	15
5.1.3	Sequence Diagram	16
5.1.4	Activity Diagram	17
5.2	Implementation and Engineering Practices	19
5.2.1	Incremental Design	19
5.2.2	Refactoring	19
5.2.3	User Database and Persistence	20
5.2.4	Test-Driven Development	21
5.2.5	Unit Testing & Integration Testing	21
5.2.6	Collaboration	23
6	Discussion	23
6.1	Cross-Team Collaboration	23
6.2	Module Development Experience	24
6.3	Requirement Maturity	25
7	Conclusion	25
7.1	Project Summary	25
7.2	Lessons Learned and Future Actions	26
8	Acknowledgements	27
8.1	Use of Generative AI	27
9	Appendix	27
9.1	Requirements Tables	27
9.1.1	Functional Requirements	27
9.1.2	Non-Functional Requirements	28
9.2	UML Class Diagram of the entire application	29
9.3	Entity-Relationship (ER) Diagram of the User Domain	30
9.4	Styleguide	30
9.5	Personas	31
9.6	Initial UML Class Diagram of Shop module	36

9.7	Noun Verb Analysis	36
9.8	Supplementary Materials	36

Contributions

Source Code Repository: [LINK](#)

Sprint	Contributions
Sprint 0	• All members: Product backlog development, requirements specification, user story creation and prioritization. • All members: Preliminary analytical documents including noun-verb analysis and CRC-cards; established Definition of Done and Definition of Ready. • Martin: Was chosen as the group leader. • Elias: Was chosen as the Scrum master.
Sprint 1	• All members: GitHub repository setup and communication channels via Jira, Messenger, and Discord. Reached initial agreement with other groups on project structure and data protocols. • All members: Individual persona drafts (customer and warehouse worker profiles).
Sprint 2	• Mikkel: AXAML project setup, directory structure, page template, and full Product Page (PP) layout including grid and card UserControls. • Elias: Initial UML diagram of the proposed project architecture. • Yaroslav & Martin (<i>pair programming</i>): All global state management tasks - class setup, accessibility across views, basket/recently viewed, and add/remove/get/reset functions. • Christian & Yaroslav (<i>pair programming</i>): PIP initial layout population and variable display. • Martin & Ziad (<i>pair programming</i>): Basket class, add/remove/clear/total functionality, checkout UserControl setup, layout, and order number display on completion. • Ziad: GitLab migration and ensuring all members had access.
Sprint 3	• Elias: Website header (with navigation structure and search bar integration) and navigation panel. • Christian: 404 error page. • Mikkel: Grid product view, FAQ page, feedback/contact form, about page, info page structure, and PowerPoint for the midway presentation. • Yaroslav: Simple search, previously viewed, favorites/save for later, and global state management for cart items and breadcrumb navigation with JSON persistence. • Martin & Ziad (<i>pair programming</i>): Basket UI rework, multiple payment methods, order confirmation page, mini-basket hover dropdown, guest checkout and order tracking view.
Sprint 4	• Yaroslav: Download order invoice button (UI, functionality carried to Sprint 5). • Christian: 404 error page (completed). Research into MVVM architecture in preparation for Sprint 5. • Elias: Research into MVVM architecture (sprint cut short due to illness). • Mikkel & Ziad: Consolidated report content from three separate sources into a single document and translated it to English. • Martin: Code cleanup, bug fixing, and translation of the user-facing UI from English to Danish. • All members: Collectively decided on MVVM as the target architecture for Sprint 5.

Continued on next page

Sprint	Contributions
Sprint 5	• All members (<i>full-day session</i>): Designed the new MVVM codebase UML together before implementation began. • Christian: Implemented the basket MVVM structure, enabling the project-wide MVVM refactoring and refactoring the remaining files which weren't assigned. • Mikkel & Ziad (<i>pair programming</i>): Landing page implementation and MVVM refactoring of their respective files. • Martin & Yaroslav (<i>pair programming</i>): Code cleanup, MVVM refactoring of their files, and implementation of sort and filter features. • Elias: ER diagram to help design the user database and SQL for table creation; collected and structured team personas into a single document.
Sprint 6	• All members: User database setup completed together as a group. • Elias: User profile dashboard UI, back-to-top button, revised ER diagram. • Christian: Stock status, unit tests (checkout and PIP), PIP back button. • Mikkel: Async PP card creation, mock data service, report writing (section 5.2). • Yaroslav: User login and password reset, developed using Test-Driven Development. • Martin: Stock reservation, product reviews. • Ziad: Remaining info/guide pages, report writing (sections 4.1-4.3), OMS API. • Christian, Elias: Report writing (sections 1.1-1.2 and 2.1-2.2 respectively).
Sprint 7	• Elias: UserDashboard data population, synced favourites and recently viewed to database, finished report sections 2 and 8.1. • Christian: Unit test (Header), fix of NumericUpDown bug, report sections 1.1-1.2 and 5.1. • Christian, Elias: moved authenticationService to domain, minor structural changes. • Mikkel: Integration test (DomainService), fix of search bug, report section 5.2. • Yaroslav: Unit tests (DomainService TotalPrice and ShippingPrice), report sections 3.1-3.2. • Martin: Integration test (Landing Page), report sections 6.1-6.3. • Ziad: Unit test (Order Confirmation), report sections 4.1-4.3. • All members: Final report review; sections 7.1-7.2 written collectively.

Table 1: Overview of team T4G1 contributions per sprint.

1 Introduction

Arnes Elektronik is a medium-sized Danish electronics store founded in 2003. The company sells products such as desktop PCs, laptops, and monitors, and has recently branched out to include PC components and provide helpful guides for customers. These include topics such as "How to change your CPU", "The difference between display types" and others.

As a result of this expansion, *Arnes Elektronik* requested a fully functional offline prototype of an e-commerce solution as a proof of concept. The solution is made up of the following modules: Shop, Product Information Management (PIM), Order Management System (OMS) and Digital Asset Management (DAM).

Due to the project being distributed between three groups rather than four, the DAM module was not implemented as part of this project. Consequently, the system operates without image support.

1.1 The Shop Module

The Shop shows products and prices to customers through a Graphical User Interface (GUI), where customers can search for products and buy them. The Shop enables a customer to put a set quantity of products into a basket, displays the total price for the items in that basket, and allows the customer to purchase these products.

The information shown in the Shop is sourced from the other modules. Product information displayed in the Shop is fetched from the PIM system, and once a customer completes the purchase, an order is submitted to the OMS.

Additionally, customers can explore various pages with helpful information, such as guides for cleaning or choosing the right product for the customers' needs.

2 Methodology

The following chapter describes the methods and processes that have been used to develop the product, ensure clear communication between groups and keep requirements up-to-date. It focuses on agile development and team collaboration, explaining which methods were applied, and why they were applied, including what value was gained from them.

2.1 Agile Development Process

To ensure frequent delivery of working software and close collaboration with other groups, the group used an agile development process. This allowed for evolving requirements, continuous feedback and managing dynamic dependencies between groups. The entire process has thus been divided into two-week iterations (sprints), each with a specific focus. After each sprint, the software should be executable and presentable to the customer, to gain feedback for continuous development. This enabled the group to adapt to changes while ensuring delivery of working software.

The Agile framework allowed iterative development with continuously updated requirements and priorities. This was mostly done at the start of each sprint during the planning phase. This ensured that each sprint had a clear and up-to-date focus that corresponded with decisions made across the team and aligned with overall project goals. Short sprints also meant that it was easy to review and evaluate the progress, and adapt the process and plan for future sprints.

The workflow was also influenced by the four P's: Product, Project, Process and People. Our understanding of the *Product* perspective helped us decide on meaningful functionality that met user needs. This was supported by artifacts such as user stories, as well as personas developed in collaboration with other groups in the team. The *Project* perspective emphasized scope, prioritization and planning through a continuously updated backlog and sprint based timelines. This included a definition of a minimal viable product (MVP) and prioritization and estimation of tasks and user stories. The *Process* perspective involved using Scrum, the agile method and engineering practices to structure development and ensure frequent deliveries of working software. Finally, the *People* perspective emphasized

communication, collaboration and shared responsibility both within the group and across the wider team.

2.1.1 Scrum

Scrum was used as the primary agile framework for managing and organizing the development process. A Scrum board was implemented using Jira to visualize work items and track progress through each sprint and the project as a whole. The board was structured around a dynamic backlog that contained all user stories, technical tasks and bug fixes needed for a finished product.

User stories were used to describe specific features from a user perspective and included a summary, assignees, status, priority, acceptance criteria and story points. These have all been designed using the Connextra template: *"As a ___ I want ___ so that ___"* making the goal of the feature very clear by identifying the user, their needs and why they matter. Depending on scope, complexity and assigned story points, work items were structured as either epics, user stories or subtasks.

To ensure consistency across work items, the group defined a general definition for when items were ready for implementation as well as when they were completed. The definition of ready included criteria such as clarity, conciseness, clearly defined acceptance criteria and a sufficient group-wide understanding of the task. The definition of done required that a task was implemented, reviewed by others, tested when appropriate and verified against its acceptance criteria.

Throughout the sprint, group members would provide individual progress reports describing work completed, work in progress, potential blockers and if any help was needed. In addition to this, each meeting started with an informal stand-up where each member would state what they are working on and demonstrate their work where applicable.

At the end of each sprint, work was reviewed and important feedback from demonstrations was taken into consideration. A sprint retrospective was also created focusing on evaluating the process and identifying actionable improvements for future sprints. Sprint metrics such as sprint velocity and burndown-charts were taken into consideration for future planning and the definition of done and/or definition of ready was updated if necessary.

2.1.2 Extreme Programming

Unlike Scrum, Extreme Programming (XP) focuses more on technical engineering practices and was used to improve code quality, collaboration, and maintainability. Selected practices from here were applied where applicable to drive the development process, ensuring that it remained efficient and that features were implemented properly.

Continuous integration

The group adopted a workflow of frequently merging small changes into a shared repository using feature branches. Merge requests required code reviews from other group members before integrating. This meant rapid feedback, avoidance of large scale merge conflicts and integration issues. Where relevant, unit and integration tests were also performed to verify that new changes did not break existing functionality.

Refactoring

The code was continuously refactored without changing its behavior. This meant improved structure and readability that was efficient without duplicate or dead code. This contributed to a more maintainable codebase and supported integration of new features.

Test-driven development

Test-driven Development (TDD) was applied to selected features, particularly in areas where reliability was critical, such as user authentication. Here, test cases were made before implementation of the feature following a "red-green-refactor" cycle: First developing a test specifying the desired behavior, then implementing just enough functionality to make the test pass, and finally refactoring and cleaning up the code to ensure it is efficient and well structured.

Pair programming

For complex or critical tasks, where knowledge sharing was important, two developers would collab-

orate at one workstation. One developer acted as the driver, writing code, while the other acted as the navigator, reviewing and providing guidance. Roles were switched when appropriate to support knowledge sharing and balanced participation.

Group Programming

Although not a formal XP practice, for significant tasks and implementations, the group would collaborate simultaneously on a single work item. This way everyone in the group could provide input and gain a coherent understanding of the task, meaning a shared understanding and consistency in implementation.

Collective Code Ownership

Every group member had authority over, and was responsible for, the entire codebase. This was supported by code reviews, shared coding standards and collaborative engineering practices. This ensured flexibility, accountability and encouraged collaboration and knowledge sharing.

2.2 Requirements

Requirement validation is the process of checking that the captured requirements describe the system in the way the customer actually wants [1]. The literature lists three techniques, namely: requirements reviews, prototyping, and test-case generation [1].

The requirements engineering process for the Shop subsystem followed the method taught through the courses attended alongside the semester project. The process therefore moved through elicitation, specification, validation and then ongoing management, which is in line with the Requirements Engineering activities described in the Fundamentals of Software Engineering (FSE) course. The literature also recognizes that agile development teams do not run a formal change management process, instead the group used an informal process, where changes were prioritized and absorbed into the next iteration [1].

2.3 Team Organization

The team organization was inspired by roles presented by Scrum, but adapted to fit the structure of the project, which involved multiple collaborating groups. These roles included the **product owner**, the **Scrum master** and the **group leader**.

The group collectively acted as the product owner, defining artifacts such as user stories, determining priority and acceptance criteria, and deciding what items would be selected for a sprint. This also included how many story points each work item should have and deciding if a completed story met the acceptance criteria.

The Scrum master was responsible for supporting the Scrum process within the group. This included organizing internal meetings with daily stand-ups, facilitating sprint planning and evaluation and overseeing the entire development process. In addition, they were also responsible for documenting meetings including decisions in the group and the team, feedback from demonstrated work and general planning.

The group leader was responsible for coordinating with the other groups in the team. Their role was to ensure constant up-to-date communication and decide on important factors such as APIs, requirements between systems and shared models and resolve common issues in the team.

To coordinate between groups in the team, group leaders and Scrum masters met weekly. These meetings served as a space to present progress, resolve questions or integration issues, identify dependencies and blockers and ensure alignment on architectural decisions. This helped reduce misunderstanding between groups and ensured consistency in the overall system with each group working to align with expectations from others in the team.

3 Scrum Reports

This chapter describes how the group worked with Scrum during the project. The focus is on how the backlog was handled, how work was planned and followed up during the sprints and how the group

adjusted its process along the way.

3.1 Product Backlog and Definition of Done

The Product Backlog was used as the main overview of the work in the project. It contained user stories, technical tasks, bug fixes and report-related tasks. The backlog was updated throughout the whole development process. In the beginning, the backlog was created from the project description, and the group's internal discussions about what the Shop module needed to support in the form of user stories. Later, the backlog was moved into Jira, where it became easier to assign issues, estimate them, and track their progress across the sprints.

Most of the backlog items were written as user stories using the Connextra format, with acceptance criteria, priorities, and story points attached. This gave the group a shared understanding of what each feature was supposed to do, which features were needed, and which were outside of the project's scope. Besides outlining user-facing features, the backlog was used for project setup, Model-View-ViewModel (MVVM) refactoring and unit testing.

The priorities and story points were estimated jointly during the group meetings, especially in the first meeting where the initial user stories were created. Priority was mainly based on how important a feature was for the MVP, meaning that the group discussed whether the feature was necessary for the shop to function in a meaningful way or whether it could be postponed to a later sprint. Story points were estimated using the Fibonacci sequence, and were mainly based on the expected amount of work, which in practice depended on both complexity and estimated implementation time. These discussions were usually straightforward, since the group often agreed on the relative importance and size of the tasks, but if uncertainty arose, views were discussed briefly until a common decision was reached. Technical tasks such as refactoring, setup and testing were estimated in the same way as user-facing features.

The backlog was continuously adjusted throughout the project, where tasks were added and some stories were discarded as they were non-essential to the project's completion. During sprint planning, the group looked at the backlog as a whole and chose the stories that were the most important, fit together well, or made the most sense to complete in the same sprint. The group tried to avoid or prioritize blockers to not slow down the development process.

The Definition of Done was both formal and practical. In practice, a feature was considered done when it met its acceptance criteria, had been reviewed and approved by at least two group members via GitLab merge requests, and was merged into the main branch. This requirement was not just procedural - it provided a consistent point at which code could be explained, questioned, and improved before becoming part of the shared codebase. The prior agreement to divide implementation work along class and feature boundaries, guided by the UML produced at the start of the project, meant that parallel development rarely produced conflicting changes, and the few merge conflicts that did arise were straightforward to resolve.

3.2 Sprint Summaries

The project was carried out in a series of sprints, which gave the group a clear way to organize and plan the work. The group usually met twice a week, on Tuesdays and Thursdays with some meetings being online. Thursday meetings were especially valuable because they included supervisor feedback and cross-group team meetings. These meetings gave the group a fixed point for discussing progress, blockers, questions and next steps.

Sprint planning was done together during meetings. In the first sprints, the group was still trying to understand how much work could realistically be completed in one sprint. During subsequent sprints, the workload was adjusted to account for group velocity and individual capacity. Following this pattern, a better sense of capacity was gained making planning more realistic and allowing for a better estimation of velocity.

Sprint 0

During this sprint, the group held the first meeting where the group leader and the Scrum master were chosen. A template for the backlog was established. A cross-group meeting with the other two groups

from our team was held where main requirements for the product were outlined and the groups agreed on the mode of communication between the different parts of the project. Here it was also decided to exclude the DAM from the project, since the other modules were deemed more important. After the basic requirements were collectively established between groups, the group started Noun-verb analysis, (as seen in appendix 9.7) and drafted Class-Responsibility-Collaboration cards (CRC cards) from this analysis.

Sprint 1

Here, focus was on establishing the foundation for the rest of the project rather than implementing complete shop functionality. During this sprint, the group created the first version of the product backlog, categorized and prioritized user stories. Then aligned the overall product vision, and finished the preliminary analysis work such as the Noun-verb analysis (as seen in appendix 9.7), CRC cards, and began work on the program specification.

Sprint 2

During this sprint the project properly started, by setting up the project structure, moving the project to GitLab, creating the first UML and implementing the core parts of the shop such as the product page, product information page, basket, checkout, and global state management. This sprint thus provided a working foundation to build upon.

Sprint 3

During this sprint, the group focused on user interface and customer-facing functionality. Here features such as search, favorites, previously viewed products, navigation, helpful pages, FAQ, order confirmation, payment options and mini-basket behaviour were added. The sprint also showed that the group became better at planning and executing sprints, since much more functionality was completed than in the earlier sprint and the workload was more evenly distributed throughout the sprint, as shown in Figure 1.

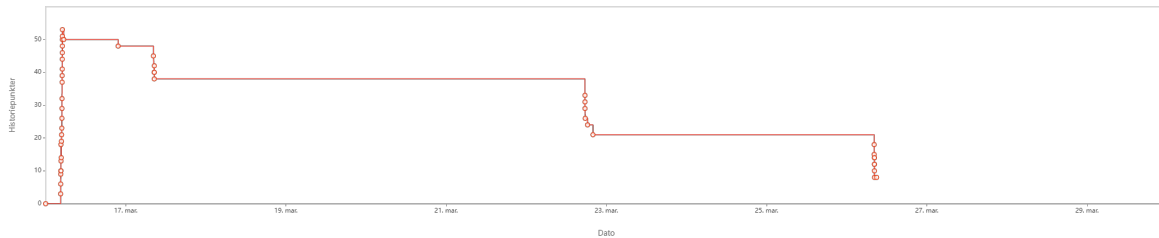


Figure 1: Burndown chart from Sprint 3

Sprint 4

This sprint was different from the others. Instead of focusing mainly on new features, the group spent much of the sprint on bug fixing, cleanup, code structure and report work. The sprint also coincided with the midway seminar, which provided external feedback that informed several decisions going forward, including the move toward MVVM. Additionally, the Easter holidays led to lower availability, resulting in this sprint producing less deliverables than usual.

Sprint 5

Here, progress was centered around the MVVM refactor, meaning that most of the sprint was spent redesigning and restructuring the existing code. This blocked some other tasks, like downloading invoice functionality, but it also made later work easier and gave the project a cleaner structure. In the second half of this sprint, the group also completed work on features such as sorting, filtering, landing page improvements and download invoice functionality.

Sprint 6

Throughout this sprint, the group worked on database creation and integration, login-related features, testing and planning integration with the other systems following a cross-group decision to implement a shared User Database. This sprint also made it clear that some parts of the project depended heavily on the other groups, creating uncertainty around the final integration.

Sprint 7

The main focus was on fixing the remaining bugs, adding a few more unit and integration tests and completing the report. Furthermore, the final presentation was developed and demonstrated, providing valuable feedback for the finalization of the project. Final changes to the codebase structure and implementation of user database functionality were also added. Compared to the earlier sprints, the work in this sprint was less about introducing new functionality and more about polishing the existing solution, making sure the application ran as expected and preparing the final delivery.

Across the project, sprint planning, reviews, and retrospectives gave the group a practical way to plan work, follow progress, and adjust along the way. In the early sprints, this mostly helped with understanding how much work could realistically be completed and establish the basic shop structure. During intermediate sprints, the focus shifted toward expanding functionality and improving the architecture, especially through the MVVM refactor. In the final sprints, the discussions were more about testing, blockers, dependencies on the other groups, integration, bug fixing and finishing the report. Overall, the sprint process helped the group keep the work organized, track and maintain steady progress throughout the project.

4 Requirements

This chapter describes how the Shop module's requirements were derived, specified, and validated. It covers the elicitation techniques used in the absence of a live customer, the cross-system contracts negotiated with the OMS and PIM groups and the resulting functional and non-functional requirement specifications.

4.1 Requirements Elicitation

Because *Arnes Elektronik* is a fictional company provided by *Hesehus*, there was no live "on-site customer" to draw from, and other elicitation techniques had to be used instead. The primary technique relied on was *Reading* related material. The group was given kick-off material by *Hesehus*, including a Project description, Case Presentation and the physical briefing at *Hesehus*' facilities. This provided written requirements for *Arnes Elektronik* for each subsystem, from which the Shop's responsibilities were extracted. These included exposing products and prices through a GUI, supporting search, basket, checkout and pulling data from PIM, media from DAM and submitting orders to the OMS. DAM was not relevant for the group's scope, and therefore no requirements were drawn up in relation to it. In addition, established webshops were examined; primarily for their features, visual layout, checkout funnel and the overall navigation flow. This was used to gain insight and inspiration, to improve the Shop's user experience.

The second technique heavily relied on active stakeholder participation, which was applied across two distinct stakeholder groups [1]. The first consisted of the group itself together with the supervisor. The group acted as a proxy product owner and proxy customer in order to write the user stories, define acceptance criteria, and validate the scope. The supervisor gave feedback at each weekly supervisor meeting, and at the end of each sprint. For example, at the end of Sprint 1, stakeholder feedback included elements such as making acceptance criteria measurable, keeping the monolithic architecture for the MVP and dedicating a fixed day to closing one sprint and starting the next. This feedback actively shaped how the group worked from that point on. The second stakeholder group was the other groups in Team 4, OMS and PIM, whose modules the Shop had to integrate with. As early as Sprint 0, shared agreements and requirements for the architecture, data structures and protocols were produced across the three groups.

Beyond the two methods of elicitation mentioned above, personas and scenarios were also used to make abstract users concrete. Together with the other groups in Team 4, seven personas were developed, as seen in appendix 9.5: three customer-side, three operator-side personas and Arne, the founder and primary stakeholder. These personas directly shaped many of our design decisions in the shop, for instance: a customer wanting easy access to guides and the ability to compare products, a warehouse employee being able to read and edit an order's details, or Arne wanting a clean checkout flow and an admin view for managing the site.

Before the requirements themselves were written, the elicited material was also passed through a lightweight object-oriented analysis, consisting of a Noun-verb analysis (as seen in appendix 9.7) and CRC cards. This served as a stepping stone from the raw case material to the functional requirements and the user stories that followed.

4.2 System Interfaces and Integration

The course material at hand explicitly left architecture, protocol and data formats undefined and states that the most important task in the project is integrating with the other subsystems. The Shop was initially purely a frontend GUI; owning no data of its own, and was simply a consumer and exposor of the other external systems. Therefore, defining interfaces with the other groups was treated as a top priority and primary requirement.

During Sprint 0, each group developed a high-level requirements list covering its own program and what it expected from the two other groups in T4. The three documents were exchanged and discussed among the groups, which allowed the team to reach a consistent mutual agreement on the programs' requirements, ensuring that all three groups were working towards a collaborative system. These documents were of course provisional, and were refined in the following sprints, through the weekly group meetings, as each module evolved.

Shop → OMS (T4G2): Orders and stock

The OMS requirements document requires their group to expose three endpoints that the Shop can call. `CreateOrder(Dictionary<productID, amount>, customerID)` accepts a finished order, and returns a boolean indicating whether the order was successfully created. `GetCustomerOrders(customerID)` returns the order history for a given customer as a JSON array of `Order` objects, each carrying `trackingNumber`, `soldPrice`, a product dictionary, and an `isShipped` flag. The document also states that the OMS will not store customer identity or product information.

Shop → PIM (T4G3): Product data

The PIM requirements document designates PIM as the source of product information, and obliges the group to expose product data through a JSON-based API. The contract demands the ID, name, price and category fields at minimum, leaving the images and description fields as optional. Specific endpoint names were to be finalized in the group meeting, and thus their document only defined the shape of the data and the fact that PIM is the source of truth.

4.3 Requirement Specification

The requirements for the Shop system are, after the above-mentioned processes, specified in two forms; functional and non functional requirements. To derive the requirements systematically, the case material and the elicited stakeholder input were read with two guiding questions in mind: *what* must the Shop be able to do, and *how* must it do it. The first question produced the functional requirements and the second produced the non-functional ones. As a concrete example, the Hesehus case description states that the Shop must expose products and prices through a GUI and support search, basket, checkout and submitting orders to the OMS. Applying the *what*-question to this passage directly led to functional requirements such as F1, F2, F4 and F5 as seen in Table 2. Likewise, applying the *how*-question to the same passage, combined with the integration constraints negotiated with OMS and PIM, led to non-functional requirements such as NF3 as seen in Table 3. Other non-functional requirements were not derived from the case description at all, but from the personas and target user analysis, such as NF5. This originates there rather than from the *Hesehus* material.

See Appendix 9.1 for the full tables.

Table 2: Preview of first five entries for Functional Requirements.

Id	Name	Description
F1	Product display	The system shall display products to the customer, with product data sourced from PIM.
F2	Product discovery	The system shall support product discovery through browsing, category filtering, sorting, and free-text search.
F3	Product detail view	The system shall provide a detailed product view, including name, description, price, category, reviews, and live stock status.
F4	Basket management	The system shall allow customers to add products to a basket, adjust quantities, and remove items, with the basket reflecting changes immediately.
F5	Checkout and order submission	The system shall allow customers to complete a purchase by entering delivery and payment information, and shall submit the resulting order to OMS.

Table 3: Preview of first five entries for Non-Functional Requirements.

Id	Name	Description
NF1	Restricted initial load	The system shall load only a restricted number of products on initial render, fetching the remainder on demand through pagination or infinite scrolling.
NF2	Loading feedback	The system shall display visible loading feedback, such as a spinner or progress bar, while products are being fetched.
NF3	Integration stability	The system shall integrate with PIM and OMS through stable agreed-upon contracts.
NF4	Definition of Done	The system shall be developed in accordance with the team’s Definition of Done: every delivered story passes its acceptance criteria, is reviewed by another group member, and has all unit tests passing.
NF5	Learnability	The system shall allow a first-time user to complete the core flow: discovering a product, viewing its details, adding it to the basket, and placing an order, without external instruction.

In addition to the requirements above, user stories, accompanied by acceptance criteria, story-point estimates and priority ratings were also used. The user stories drove the sprint planning, while the overall requirements described the higher-level expectations the Shop had to meet.

Every backlog item was given a priority rating that was intended to be used as the main signal during sprint planning, where the user stories with the highest priority would be focused on first, and the rest deferred. In practice, this is not what happened. Once sprint planning actually began, the priority field was rarely relied upon on its own; instead, a fresh sweep through the entire backlog was taken, reading each story, weighing it against the sprint duration, recent feedback and the state of the module, to decide whether or not to complete that user story in the respective sprint. Thereby, the priority ratings became, at best, a secondary input. This is a recurring observation in agile practice, where “changing requirements” and “vague requirements” are among the top challenges [1]. This gap between the expected prioritization method and the actual method used is further reflected on in the discussion (Chapter 6).

4.4 Requirements Validation

Of the three requirements validation techniques; requirements reviews, prototyping and test-case generation - the project naturally leaned towards the first. Prototyping was not used and test-case generation only existed implicitly through the acceptance criteria, which were treated more as a Definition of Done gate than as a validation tool, and even this was dropped towards the end of the project, where the group instead simply discussed and agreed whether a story was done.

The main validation activity was cross-group review of the requirements documents. Each of the three groups read the other two groups' documents and any disagreements or questions were carried into the weekly meetings between the group leader and the Scrum master of each group. This is a lightweight version of the requirements review process, wherein the reading was done individually, discussion was held in person, and decisions were recorded as adjustments.

A concrete example of this loop is the `productID` type. Originally the `productID` was defined as a `string`, which was discussed cross-group and subsequently changed to an integer to align across the three modules.

4.5 Ongoing Management

Requirements changes did happen, and were handled informally through negotiation and then re-evaluation of the user stories, instead of changing the functional and non-functional requirements. This meant the project mapped directly onto the agile pattern, with both its strengths and its costs. An example of this is that PIM initially planned to implement a JSON-based API, but later on, when the implementation matured, the team settled on having a shared project directory, meaning the JSON-file requirements became obsolete. Instead, the systems would expose data through the shared data objects. Notably, the original requirements document was never retracted or re-written; it simply stopped being true. A second example is the Customer table: OMS initially stated it would not manage customer information, then began planning it, then handed it back to the Shop, a real change to the inter-group contract that produced no updated requirements document either.

What was done instead was to edit the user stories and acceptance criteria through Jira, to reflect the updated state. This is the essential agile mode of change management: prioritize, absorb, and move on. The cost is that the requirements specification and the system implementation do not stay in step [1]. The original requirements were therefore frozen and never re-opened after the user stories had been written. The live picture of what the system was supposed to do lived only in Jira and in the team's collective memory.

5 Technical

This chapter details the system architecture, design decisions, and software engineering practices implemented during the development of the Shop module.

5.1 System Modeling and UML

The UML diagrams were used to describe both the structure and behavior of the Shop module. The diagrams helped make the design easier to understand before, during and after implementation. The diagrams included in the project consist of class diagrams, package structures within the class diagrams, sequence diagrams, and activity diagrams. Together, the diagrams visualize how the system is built, how the different classes work together, and the overall purchase flow.

5.1.1 Class Diagram

The class diagram is used to describe the main structure of the application. It shows the classes, their properties, methods and the relationships between the classes.

The [UML Class Diagram of the entire application](#) shows that the application is built around the MVVM pattern. This means that the user interface is separated from the logic behind it, which is significantly different from the [Initial UML Class Diagram of Shop module](#). The main components of the diagram

built around MVVM are the views (purple), ViewModels (red), domain logic classes (yellow), and data logic classes (cyan). The View classes, implemented as code-behind files, do not contain logic. Instead, they forward UI state and user interactions to their respective ViewModels. The ViewModels contain commands and properties used by the Views. For example, ProductPageViewModel handles filtering, sorting and product collections, while ProductPageView is mainly responsible for displaying the page. This structure is reflected in the implementation, where ViewModels expose properties and commands that are bound bi-directionally to the Views, meaning changes in either the UI or the ViewModel are automatically reflected in the other.

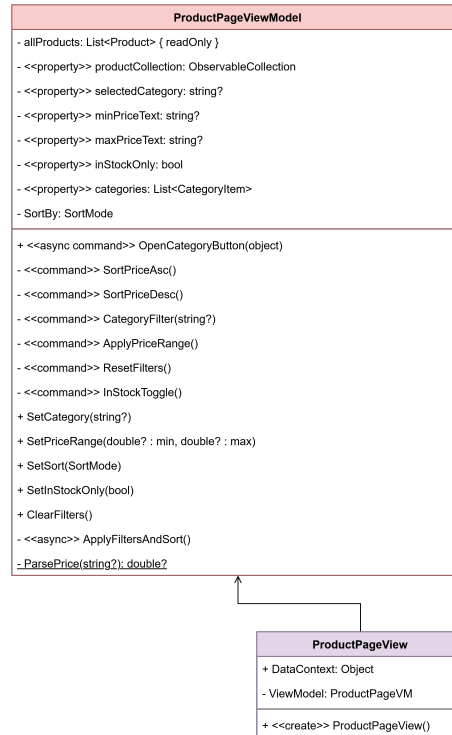


Figure 2: ProductPage View and ViewModel in UML Class Diagram

The class diagram was used to plan how the code should be divided. It made it easier to see which classes were responsible for user interface, page logic and data handling.

5.1.2 Package Structure in the Class Diagram

The class diagram also includes a package-like structure because several classes are grouped into larger areas. Examples of grouped areas include: **State Management**, **Basket Handling**, **FAQ Page Setup**, and **Cleaning Guide Setup**. This grouping was useful in the project because it gave a better overview of the system. Instead of looking at all classes as one large structure, the diagram shows which classes belong together. The package-like structure also helped show the dependencies between the different parts of the system. The State Management Package is important because the shop needs to remember multiple states while the user moves between pages.

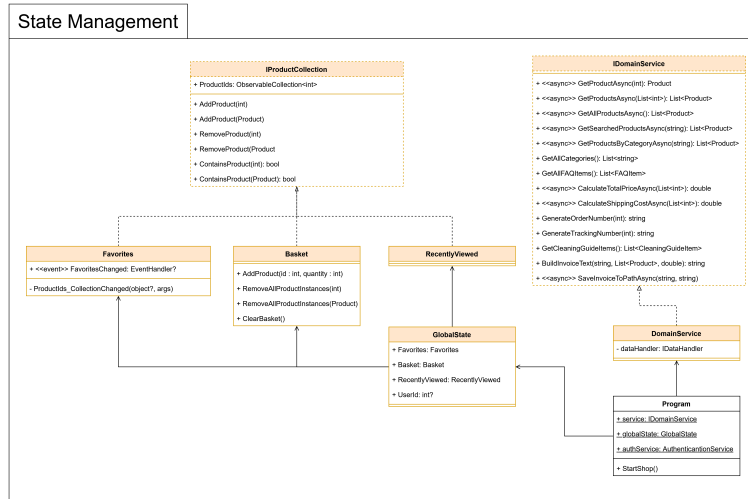


Figure 3: State Management as Package

A key class in this area is the GlobalState class, which stores information related to the basket, favorites, recently viewed items, and the user. In the implementation, this shared state is handled through a central class that can be accessed across multiple ViewModels. For example, if a product is added to the basket from the product information page, the mini basket and basket page can access the same basket information. The Favorites and RecentlyViewed classes are also part of the shared state. These classes help the system remember products the user has liked or viewed.

5.1.3 Sequence Diagram

The sequence diagram was used to describe the interactions and exchanges between the OMS, Shop and PIM.

The diagram shows the order of communication between these systems during a customer purchase. It starts when the customer enters the front page and continues until the order is confirmed.

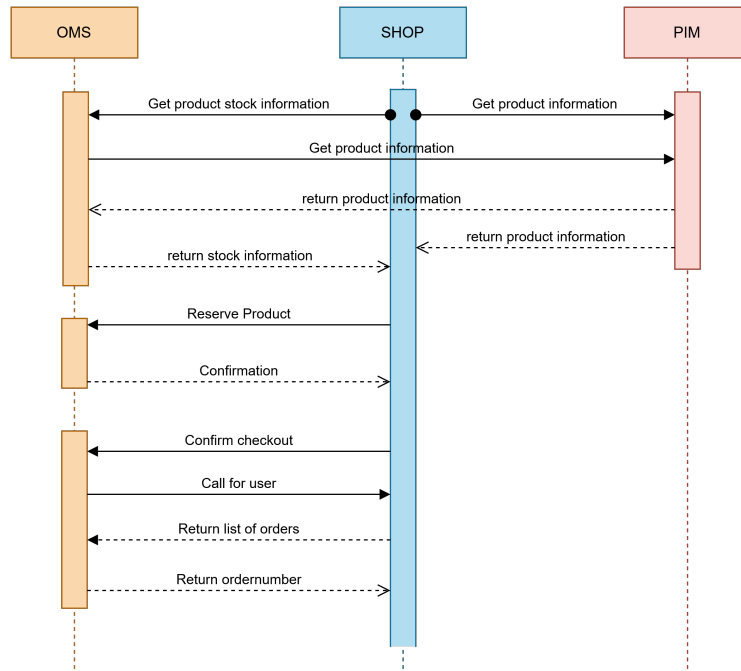


Figure 4: Sequence Diagram illustrating the intended cross-module communication flow

The sequence diagram is useful because it shows the communication between the modules. It makes it clear which system is responsible for each step. In the diagram, Shop is the central modules. Shop handles the customer-facing part of the system. The PIM is used when the system needs product information. OMS is used when the system needs stock information or the order needs to be confirmed.

5.1.4 Activity Diagram

The activity diagram is used to describe the flow of the Shop from the users perspective. It focuses on the actions and decisions in the system, rather than the communication between modules as shown in Figure 4.

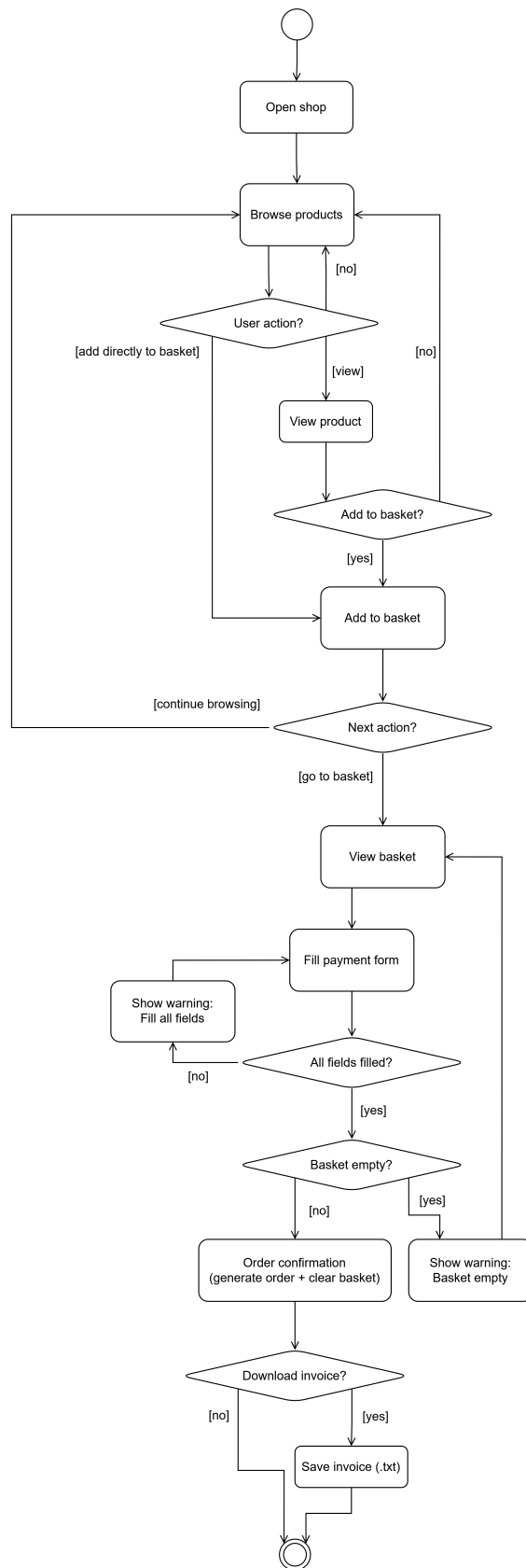


Figure 5: Activity Diagram

The diagram illustrates how a user interacts with the Shop, starting by opening the Shop and continuing by browsing products, viewing product details, adding items to the basket and proceeding to checkout. Decision points are included to show alternative paths, for example whether the user continues shopping or completes the purchase. This makes it possible to see the different paths a user can take through the system. The activity diagram was useful in the project because it provided an overview of the user flow.

5.2 Implementation and Engineering Practices

This section describes the engineering practices applied during the development of the Shop module. It covers how the system was built incrementally, the structural refactoring undertaken midway through the project, the team's experience with TDD, the testing strategy applied at both unit and integration level and the collaborative practices used across the development process.

5.2.1 Incremental Design

The Shop module was built incrementally across seven sprints, with each sprint deliberately layering new functionality onto a stable foundation rather than attempting to deliver a complete system all at once. This approach was established from the outset through a key structural decision made in Sprint 2: the `MainWindow` was designed as a persistent shell containing a header, a content area, and a footer, where only the content area changes as the user navigates the application. Each page in the shop was implemented as a `UserControl` that slots into this area, meaning new pages could be developed and added without touching the surrounding layout.

This shell-first approach allowed the group to have a navigable, visually complete application early in the process - one with real pages that simply did not yet do anything. The Product Page is a representative example of how the incremental model played out across the project. It was introduced in Sprint 2 as a grid of product cards backed by mock data. Over subsequent sprints, search functionality, category filtering, and sorting were each added as self-contained extensions to the existing page. None of these additions required the page to be reconceived; they extended it.

A similar pattern emerged with shared infrastructure. Early in development, page navigation was handled locally within each view, which resulted in repeated logic spread across the codebase. As the number of views grew, this became unsustainable. The `PageHandler` - a static singleton accessible from all scripts - was introduced to centralize navigation under a single responsibility. This not only eliminated duplication but also made the navigation behaviour consistent and easier to reason about as the application scaled.

5.2.2 Refactoring

The most substantial structural change in the project came in Sprint 5, when the entire codebase was refactored to follow the MVVM pattern using the `CommunityToolkit.Mvvm` library on .NET 10.

Prior to the refactor, the application's logic was largely contained in the code-behind files of each Avalonia view. While this was sufficient during the initial sprints, it became increasingly problematic as the codebase grew: business logic was entangled with presentation code, making it difficult to modify one without risking the other, and the lack of a consistent structural boundary made the project harder to navigate as a team. When it became clear that existing structure could not cleanly support the scale and complexity the codebase had reached, a full refactor was undertaken - driven by the poor code quality and high coupling, as illustrated in Listing 1.

```

1 public HeaderView()
2 {
3     InitializeComponent();
4     // View-level wiring happens directly in code-behind
5     MiniBasketPopup.PlacementTarget = BasketButton;
6     MiniBasket.RequestNavigateToBasket += (_, _) => SwitchToBasket_Click(null, new RoutedEventArgs());
7 }
8
9 private void OnSearchTextChanged(object sender, TextChangedEventArgs e)
10 {
11     // Directly reads UI state and performs filtering logic here
12     ...
13     if (SearchBox.Text == "")
14     {
15         // Resets navigation state from inside the view
16         ...
17     }
18     // Builds a new page instance instead of delegating to a ViewModel
19     ...
20 }
21
22 public void SwitchToBasket_Click(object? sender, RoutedEventArgs e)
23 {
24     // Navigation, event wiring, and page creation are mixed together here
25     ...
26     MainWindow.Instance.PageContainer.Content = basketPage;
27 }

```

Listing 1: Excerpt from the HeaderView code-behind script prior to MVVM refactoring

The refactored structure assigns a dedicated ViewModel to each page. ViewModels hold the state and logic for their respective views, exposing properties and commands through CommunityToolkit.Mvvm’s [ObservableProperty] and [RelayCommand] attributes. These annotations generate the necessary property change notification boilerplate automatically, keeping the ViewModels concise. The AXAML views bind directly to these properties and commands, leaving the views responsible only for rendering state - not managing it.

Where functionality is shared across multiple ViewModels, it was extracted into centralized components to avoid duplication. The PageHandler and the global state management class are both examples of this: they are shared singletons that any part of the application can access, consistent with the **Don’t Repeat Yourself** (DRY) principle. The refactor redistributed responsibilities across these boundaries rather than introducing them all from scratch, which meant the structural work, while extensive, had a clearly defined scope.

5.2.3 User Database and Persistence

To support user authentication and persistent user-specific functionality, a SQL-based user database was implemented. This database was more specifically introduced to allow information such as user accounts, favourites, basket contents, addresses, and recently viewed products to persist between sessions. This made it possible for logged-in users to maintain a consistent experience even after restarting the application. Furthermore, SQL was chosen because the relational structure of the user data naturally fit a normalised database model with clearly defined relationships between entities.

The structure was modeled using an [Entity-Relationship \(ER\) Diagram of the User Domain](#), which defines the relationships between entities in the system such as User, Address, BasketItem, FavouriteItem, and RecentlyViewedItem. Primary and foreign key relationships were used to connect user-specific entities such as addresses, basket items, favourite products, and recently viewed items back to the corresponding user account. Modeling the database structure before implementation helped establish a clearer understanding of how user-related data should be organised and connected throughout the system, instead of having to depend on trial-and-error.

The User entity stores core account information such as username, password, email, phone number, role and name. Address information was separated into its own table and connected through the UserAddress relationship to maintain a normalised database structure. The additional tables are used

to manage related user data. For example, `BasketItem` stores products currently added to the user's basket, `FavouriteItem` stores favoured products, and `RecentlyViewedItem` tracks products viewed by the user alongside a timestamp. The implementation required support for common CRUD operations, including user creation during registration, retrieval of persisted data during login, updating basket and favourite data during runtime, and synchronisation of user state back to the database. Queries were used to retrieve user-specific data during authentication and application startup, allowing the persisted state to be reconstructed dynamically.

The database was integrated into the application alongside the existing `GlobalState` system. For users who are not logged in, the application continues to use the in-memory `GlobalState` class to manage features such as basket contents, favourites and recently viewed products during the current session. However, once a user logs in, the SQL database replaces this temporary state by loading and storing the user's data directly from the database.

The `ViewModels` do not communicate directly with the SQL database. Instead, database operations are handled through dedicated data-layer classes and services, allowing the `ViewModels` to remain independent of persistence logic. This separation is consistent with the MVVM architecture used throughout the project and improved maintainability by isolating data access from presentation logic. The persistence layer was primarily triggered during authentication events and state-changing operations, such as adding products to the basket, updating favourites, or viewing products.

This approach allowed the application to support both guest users and authenticated users without requiring separate implementations of the Shop.

Security was also considered as part of the database implementation. Since the database handles sensitive user information such as passwords and personal data, passwords are not stored in plain text. Instead, each password is processed through a hashing and salting mechanism before being stored, ensuring that raw credentials are never persisted. Furthermore, all database queries are constructed using parameterized inputs, which prevents SQL injection attacks by separating query logic from user-supplied data. These measures were treated as baseline requirements rather than optional additions, given that the system handles real authentication flows.

5.2.4 Test-Driven Development

TDD was applied to specific features during the project, most notably the user login flow and the product reviews functionality, both of which were implemented in the later sprints. For these features, test cases were written prior to the implementation, following the standard red-green-refactor cycle.

The experience brought into focus what TDD demands in practice. Writing a test before the implementation requires a precise specification of the expected behaviour - including edge cases and failure states - before the relevant classes and methods exist. This shift in thinking proved to be the primary source of friction. A common situation was encountering a minor edge case, such as a potential null reference, and weighing the cost of writing a full test fixture against simply adding a guard clause. The overhead of a complete test - with setup, assertion, and teardown - can feel disproportionate when the fix itself is a single line. The group noted this tension throughout the TDD sessions.

The potential of the approach, however, was also evident. A comprehensive test suite establishes a safety net that is particularly valuable in incremental development: when new features are added or existing code is refactored, failing tests pinpoint regressions immediately and precisely. This benefit is discussed further in Section 6.

5.2.5 Unit Testing & Integration Testing

Integration testing was given architectural consideration throughout the project, even before it could be executed with real data. To enable both isolated development and eventual cross-system integration, the groups agreed on a monolithic repository structure in which each module - Shop, PIM, and OMS - exists as its own sub-repository while also being accessible from a shared host project. This structure allows each group to build and run their module independently, and also allows all three modules to be launched together from the host project, connected through direct references. Until the other systems'

APIs were available, the Shop used a mock data generator internally, ensuring that development and testing could proceed without external dependencies.

The integration tests targeted the boundary between the domain and data layers. `DomainServiceTest` instantiated a real `DataHandler` and exercised the full call chain through to the `MockPIM`. This was used during development, wired into the `DataHandler` through the same interface a real PIM API would eventually occupy. Two areas are covered: JSON file reading and shipping cost calculation. For the JSON tests, three cases are verified - a well-formed file (Happy Path), a malformed file raising a `JsonException` (Sad Path) and a missing file raising a `FileNotFoundException` (Missing File) as seen in Listing 2. File state is restored between tests via `SetUp` and `TearDown` to ensure isolation.

```
1 [Test, Order(1)]
2 public void HappyPath()
3 {
4     File.WriteAllText(targetFilePath, ...); // Replace with valid test file
5
6     var items = service.GetCleaningGuideItems();
7
8     Assert.That(items.Count, Is.EqualTo(1));
9     Assert.That(items[0].GuideTitle, Is.EqualTo("Test Titel"));
10 }
11
12 [Test, Order(2)]
13 public void SadPath()
14 {
15     File.WriteAllText(targetFilePath, ...); // Replace with malformed JSON
16
17     Assert.Throws<JsonException>(() => _ = service.GetCleaningGuideItems());
18 }
```

Listing 2: Integration tests for JSON file reading in `DomainServiceTest`

Beyond the integration tests, the group developed unit tests for several of the Shop's core pages, including the Product Page, the Product Information Page, the Checkout Page, the Basket, the user authentication flow, and more. The approach was to execute `ViewModel` commands directly and assert on the resulting observable state, verifying that the application behaves correctly in response to user actions, as shown in Listing 3.

```
1 [Test]
2 public void TestAddToBasketException()
3 {
4     var pipVM = new ProductInformationPageVM(product.productID);
5     pipVM.Product = null;
6     Assert.ThrowsAsync<InvalidOperationException>(
7         () => pipVM.AddToBasketCommand.ExecuteAsync(null));
8 }
9
10 [Test]
11 public void TestFavoriteAdd()
12 {
13     var pipVM = new ProductInformationPageVM(product.productID);
14     pipVM.IsFavoritesItem = false;
15
16     pipVM.FavoriteProductCommand.Execute(null);
17
18     Assert.That(pipVM.IsFavoritesItem, Is.EqualTo(true));
19 }
```

Listing 3: Unit tests for `ProductInformationPageVM`

Planned System Testing

A full cross-system test was planned but not executed within the project period. The monolith structure was established specifically to enable this. All three modules can be launched collectively from the host project, with PIM and OMS running on background threads and the Shop on the

main thread, each opening in its own window. The intent was to verify the full data flow - the Shop retrieving real product data from PIM, a completed checkout producing an order in OMS and stock levels updating accordingly. Due to time constraints in the final sprint, the groups reached a joint agreement to deprioritise the cross-system test in favour of completing individual module deliverables.

5.2.6 Collaboration

The group employed pair programming throughout the project in two distinct modes, each suited to different situations.

The first mode structured sessions around a junior-senior dynamic: the less experienced developer on a given task took control of the keyboard and drove the implementation, while the more experienced developer took a mentoring role - prompting considerations ("have you checked whether that could be null?", "didn't we implement something similar already?") without dictating the solution. This mode was particularly effective for knowledge transfer, as the person writing the code was actively making decisions rather than passively watching.

The second mode operated more as a driver-navigator arrangement: one person verbally directed what should be written and explained the reasoning, while the other typed. This was used when completing the basket and checkout flow, where two developers split responsibilities cleanly between thinking and writing, with both contributing continuously throughout the session. This worked well for implementing well-understood features quickly, where the priority was speed rather than individual learning.

A third, less formal mode - group programming - was used at key moments: the kickoff of the MVVM refactor and sessions involving merge conflicts. The whole group would gather around a shared screen with a Live Share session, allowing anyone to contribute directly. This was especially effective at the start of the MVVM refactor, where the pattern was new to everyone and a shared initial implementation gave the whole group a concrete, unified understanding of how the structure should look and work before splitting off to refactor their individual areas.

6 Discussion

6.1 Cross-Team Collaboration

The project required close collaboration between multiple groups responsible for different parts of the overall system, including the OMS, the PIM and the Shop. This task was one of the most valuable, but challenging aspects and from the outset it was essential to establish shared understandings and expectations regarding data exchange, architecture and responsibilities. Since each group was working independently on separate functionalities, establishing this early ensured effective communication and a high quality result.

A significant part of the collaboration involved jointly defining the system architecture, following Sprint 1 supervisor feedback and several discussions between the groups. During the planning phase, multiple architectural approaches were considered, including simulating web APIs by exchanging files between dedicated folders for each module in order to better reflect distributed system communication. However, due to project scope, time constraints and additional complexity, the groups ultimately decided to use a monolithic structure. The GitLab repository was therefore divided into sub-repositories for each system component, meaning that each group could manage and edit their own part of the project independently, while still allowing the other groups to view the code for integration purposes. This setup helped maintain clear ownership of each component while still allowing for close collaboration and transparency between the groups. To further support the communication between modules, the groups created shared objects in a shared sub-repository that defined the data structures used across the systems. This ensured consistency, simplified communication between modules and reduced integration issues when exchanging information between the OMS, PIM and Shop.

Coordination between groups was not always straightforward since different groups sometimes had varying ideas about implementation details and priorities, which slowed decision-making. Another challenge was synchronizing the development timelines between the different groups. Since the OMS,

the PIM and the Shop components depended on one another for integration and testing, delays or changes in one module could affect the progress of the others. On several occasions, one group needed functionality or objects that were still being developed by another group, making it difficult to fully test features independently. A concrete example of this was the repeated renegotiation of customer data ownership between the Shop and OMS, where responsibilities shifted multiple times mid-project before a final agreement was reached. From a collaborative perspective it highlighted how differences in domain understanding and initial uncertainty regarding system responsibilities could cause inter-group contracts to become outdated quickly. It also demonstrated how progress was dependent on maintaining an active dialogue between groups. However, to combat this issue and maintain alignment throughout the project, the groups held weekly meetings where progress, integration concerns and upcoming tasks were discussed. In addition, online communication was used extensively for day-to-day communication, allowing for quick clarification of issues and faster collaboration within the team.

Although the groups established a shared architecture and maintained regular communication throughout the project, the final systems were not integrated by the end of the project. This was primarily due to time constraints, differing development priorities and dependencies between modules that delayed integration testing. Since each group was simultaneously developing and refining their own modules, some functionality and shared objects were not finalized early enough to allow for integration. Moreover, a significant amount of time was spent establishing and completing the individual modules, which reduced the time available for integration. Therefore, to achieve a successful integration, the groups could have prioritized integration earlier in the project by implementing smaller integration milestones throughout the sprints rather than focusing mainly on individual module completion. Earlier finalization of shared interfaces and APIs and allocating dedicated time specifically for integration and debugging would likely have improved the chances of fully connecting the OMS, the PIM and the Shop modules before the project deadline.

Despite the initial difficulties, which reflected aspects of the 'storming' phase described in Tuckman's stages of group development [2], the groups gradually established clearer communication patterns that supported more effective collaboration. Working closely with other groups provided valuable experience in coordinating software development across multiple groups, similar to real-world development environments. It also improved group members' ability to communicate technical ideas and architectural decisions clearly and adapt the implementation to fit a shared system design.

6.2 Module Development Experience

The development of the Shop module began with brainstorming sessions in which the group drew inspiration from already established platforms to better identify the functional requirements and prioritize them based on their complexity, feasibility and relevance to the overall system. This process generally worked well, as it encouraged discussions and allowed the group to approach the project with a clear set of user stories. However, different group members also had differing opinions regarding which functionality should be included and how certain features should behave. While these disagreements occasionally slowed decision-making, they ultimately contributed to a well-evaluated set of tasks for the project and a shared understanding throughout the group.

Two patterns emerged during the implementation phase that, in hindsight, significantly shaped the trajectory of the project: the late introduction of MVVM and the delayed adoption of TDD. Both approaches proved valuable, but in both cases, the timing reduced the degree to which the group could benefit from them.

The decision not to implement MVVM from the beginning was, at the time, a pragmatic one. None of the group members had prior experience working with it and the immediate priority was to establish a functioning application as quickly as possible. The code-behind approach was familiar, easier to work with initially and sufficient for the early stages of development. What was underestimated was how rapidly the codebase would grow as additional features were introduced. Over time, the absence of a clear separation between views and application logic resulted in tighter coupling throughout the system, making the project increasingly difficult to maintain and extend. By Sprint 5, the refactor toward MVVM had become unavoidable, requiring a full sprint to be dedicated almost entirely to restructuring the architecture. In retrospect, adopting MVVM already by Sprint 2 would likely have reduced the accumulated technical debt and the long-term cost of maintaining the codebase.

A similar pattern emerged with the adoption of TDD. The group observed that TDD requires a fundamentally different development workflow, where expected behaviour is defined before the implementation itself is written. This mindset is significantly easier to establish at the beginning of a project when there are no existing habits to displace. Introducing TDD late in the project meant that the group had to balance the cognitive overhead of learning an unfamiliar methodology with the pressure of ongoing sprint deliverables. Although the value of automated testing and regression prevention became clear during implementation, the long-term advantages of TDD could not fully materialize because the methodology was applied primarily to features developed near the end of the project.

The core lesson from both experiences was ultimately the same: architectural and methodological decisions are most effective when introduced early enough to compound across the remaining time for the project. A difficult decision made early supports convenient future development, whereas the same decision made late introduces both the cost of implementation and the additional overhead of undoing previous work. This became one of the most transferable lessons gained from the development of the Shop module that the group will carry into future software projects.

6.3 Requirement Maturity

At the beginning of the project, many of the requirement specifications existed primarily as broad functional goals rather than fully detailed technical requirements. The overall purpose of the Shop module was clear, but many aspects of the final implementation, such as specific workflows, API behaviour and feature priorities, were still uncertain. This meant that several requirements reflected assumptions rather than finalized decisions about how the system should function.

The Agile development process allowed the user stories derived from the requirements to mature gradually throughout the project lifecycle. Instead of treating the initial user stories as fixed, the group continuously refined and adjusted them during sprint planning, implementation and review phases. As development progressed, both limitations and new possibilities in the implementation of the requirements became apparent, which led to adjustments in the associated user stories that had not yet been evident during the initial planning stages. Some user stories became more detailed over time, while others were simplified, reprioritized, split into smaller tasks or removed entirely based on feasibility and project scope. This was also reflected in the evolving use of feature priority ratings discussed in the [Requirements](#) section, where initially defined priorities became less rigid as integration dependencies and time constraints affected development throughout the project.

The iterative workflow was especially useful because it allowed the group to continuously test assumptions instead of fixing requirements too early, reflecting aspects of the Build-Measure-Learn cycle commonly associated with Agile and Lean development [3]. Through early prototypes and partial implementations, issues related to usability, technical limitations and system integration became visible and led to adjustments in the user stories. This gradually reduced uncertainty and helped ensure that later decisions were based on a more realistic understanding of what the system needed and could support. At the same time, as the group gained more experience with the technologies and overall system design, the requirements naturally became more detailed and better aligned with actual implementation. Some features turned out to be more complex than initially expected, while others were simplified or deprioritized once their practical value became clearer.

Overall, the Agile and iterative workflow helped the project move step by step from initial ideas to clear and well-defined functionality. Instead of seeing changing requirements as a problem, the project showed that continuous refinement improved both the realism and quality of the final system, by making decisions based on ongoing development experience.

7 Conclusion

7.1 Project Summary

The aim of the project was to develop a customer-facing Shop module as part of a larger offline e-commerce prototype for *Arnes Elektronik*, that had to integrate and work together with the groups in Team 4. The final delivery is a functional Shop, structured around the MVVM pattern and built

in Avalonia with .NET. User data is stored in an SQL database for authenticated users, while guest sessions continue to be handled through the GlobalState class. At the module level, the Shop supports product discovery through browsing, filtering, sorting and searching. The customer can also see placeholders for the detailed product information page, basket and checkout, order confirmation and tracking and several static content pages that the customers can view. By design, product data is meant to be sourced from the PIM and orders submitted to OMS, but this has not been implemented and the Shop instead uses mock data.

Measured against the success criteria defined in Sprint 0, the prototype does not fully deliver on what it set out to do. The functional criterion required that the user can navigate the Shop and complete a full purchase, and that all functional requirements have been implemented and tested by the deadline. This criterion is only partially met. A user can move through the entire Shop and reach the end of the checkout flow, but several of the requirements that this depends on are themselves only partially in place. Looking at the functional requirements seen in Table 4, F1, F3, F5 and F6 are operational only against mock data and a non-existent OMS and PIM, so the products displayed are mock data, the reviews shown and submitted remain mock data/interactions, the order placed at checkout is never submitted and the order confirmation page that follows is itself a mock. F8 (order history) and F10 (administrator role) were not implemented, and F9 (personal information management) was carried as far as the database schema but never wired into the user interface. Several non-functional requirements, seen in Table 5, remain unverified. NF1 and NF2 were not implemented in the final delivery, and no user tests were conducted during the project. The remaining requirements are implemented at the module level and the Shop is structured to support full integration, with mock data serving as a direct substitute for live PIM and OMS connections. This makes the core flow visibly walkable from the customer's perspective, even though the system underneath does not function end-to-end.

A separate and unrelated, yet still limiting factor, is the absence of the DAM module. Since only three groups were assigned to Team 4, DAM was never developed, and the shop accordingly operated without any true image support, with the Shop module having some hardcoded images for the content pages (front page, about us, guides, etc). This was anticipated from Sprint 0 and is a deliberate scope decision rather than a shortfall in the delivery of the end-product.

Within these boundaries, the project does demonstrate numerous things. It demonstrates an internally coherent Shop module, a concrete cross-group integration contract and a codebase that follows MVVM. The codebase is thereby structured cleanly enough to support the integration work that remains and also a multi-module architecture, had the full system integration been completed. What it does not demonstrate though, is that three modules function together as a single system, and the prototype is, in that sense, only a partial proof of concept: strong on the Shop in isolation, but incomplete as a whole system.

7.2 Lessons Learned and Future Actions

One of the clearest lessons from the project was that architectural decisions should be made carefully as early as possible. Initially, the Shop was built without a formal architectural pattern, where all of the logic was placed directly in the code-behind files for the Views. This approach was chosen because of its simplicity and because it allowed the group to get the first functionality running quickly. However, it became increasingly harder to maintain as the project grew and more features were added. As discussed in Section 6.2, this ultimately required a full sprint to be dedicated to refactoring alone. Therefore in a future project, it would be better to establish the most optimal architecture as early as possible as this could have reduced later refactoring work and made the implementation process more stable from the start.

A related lesson concerns the adoption of TDD. As with MVVM, TDD was introduced late in the project, limiting the degree to which the group could benefit from it. In a future project, establishing a test strategy from the first sprint - even with simple unit tests - would build a safety net that makes later refactoring and feature additions significantly less risky.

From a cross-group perspective, the clearest lesson was that interface contracts between groups need to be treated as living documents. Several agreements changed mid-project without formal updates to the requirements, which created integration uncertainty for the Shop. In future projects, any change

to a cross-group contract should trigger an explicit review and update across all affected groups, rather than being absorbed informally.

Finally, the project reinforced that early investment in process and architecture - though costly in the short term - compounds positively across the remaining lifetime of the project. The decisions that proved most expensive were not the difficult ones made early, but the easy ones made early that had to be undone later.

8 Acknowledgements

8.1 Use of Generative AI

During development, generative AI was used as a supplementary resource to support learning, research and documentation.

AI was primarily used as a supporting tool where additional guidance was needed. This included guidance, explanations and suggestions on unresolved complex coding challenges, reducing time spent on research. This was only used as a last resort to ensure learning and an understanding of the issue. AI also provided insight into implementations outside the group's previous experience - most notably the refactoring to MVVM, where AI could supply explanations, boilerplate structures and alternative approaches, all of which accelerated the learning process. All solutions were however independently implemented and validated by the group.

AI was also used to assist in the creation of test cases, identify bugs and suggest potential refactoring improvements for existing code. Furthermore, it was used to generate mock data such as products and users, used for testing and early implementation.

Some non-functional content for the shop was also generated by AI. This included the various guides, FAQ entries and help pages. In addition, AI tools were used to assist report writing. This included grammar correction, inspiration, proofreading and rewording to improve clarity and readability.

In all cases the output generated was evaluated and adapted with a critical perspective and later validated by the group to ensure reliability. AI was therefore used as a tool and as support, rather than a substitute for independent problem solving and design decisions in the group.

References

- [1] I. Sommerville, *Software Engineering*. Harlow, England: Pearson, 10 ed., 2016. Global Edition.
- [2] Judith Stein, "Using the stages of team development." <https://hr.mit.edu/learning-topics/teams/articles/stages-development>, n.d. Accessed: 21/5-2026.
- [3] Dominic Rogers, "Build, measure, learn cycle." https://medium.com/@dominic_11011/build-measure-learn-cycle-ace388a13b4d, n.d. Accessed: 21/5-2026.
- [4] Google, "C# at google style guide." <https://google.github.io/styleguide/csharp-style.html>, n.d. Accessed: 20/5-2026.

9 Appendix

9.1 Requirements Tables

This appendix contains the full list of functional and non-functional requirements for the Shop sub-system.

9.1.1 Functional Requirements

Table 4: Functional Requirements

Id	Name	Description
F1	Product display	The system shall display products to the customer, with product data sourced from PIM.
F2	Product discovery	The system shall support product discovery through browsing, category filtering, sorting, and free-text search.
F3	Product detail view	The system shall provide a detailed product view, including name, description, price, category, reviews, and live stock status.
F4	Basket management	The system shall allow customers to add products to a basket, adjust quantities, and remove items, with the basket reflecting changes immediately.
F5	Checkout and order submission	The system shall allow customers to complete a purchase by entering delivery and payment information, and shall submit the resulting order to OMS.
F6	Checkout summary and confirmation	The system shall present a checkout summary covering basket contents, subtotal, and shipping, followed by a confirmation upon successful order submission.
F7	Customer accounts	The system shall support customer accounts with login, password management, and a personal dashboard.
F8	Order history	The system shall present the customer’s order history in the personal dashboard, with order data sourced from OMS.
F9	Personal information management	The system shall allow customers to manage personal information, including saved delivery and billing addresses, for reuse at checkout.
F10	Administrator role	The system shall support an administrator role with elevated permissions.
F11	Support content	The system shall provide static support content, including an FAQ page and helpful articles.
F12	Site navigation	The system shall provide a menu for navigation between the website content.
F13	Error handling	The system shall handle invalid or unknown routes by displaying an error page with navigation back into the site.

9.1.2 Non-Functional Requirements

Table 5: Non-Functional Requirements

Id	Name	Description
NF1	Restricted initial load	The system shall load only a restricted number of products on initial render, fetching the remainder on demand through pagination or infinite scrolling.
NF2	Loading feedback	The system shall display visible loading feedback, such as a spinner or progress bar, while products are being fetched.
NF3	Integration stability	The system shall integrate with PIM and OMS through stable agreed-upon contracts.
NF4	Definition of Done	The system shall be developed in accordance with the team’s Definition of Done: every delivered story passes its acceptance criteria, is reviewed by another group member, and has all unit tests passing.

9.3 Entity-Relationship (ER) Diagram of the User Domain

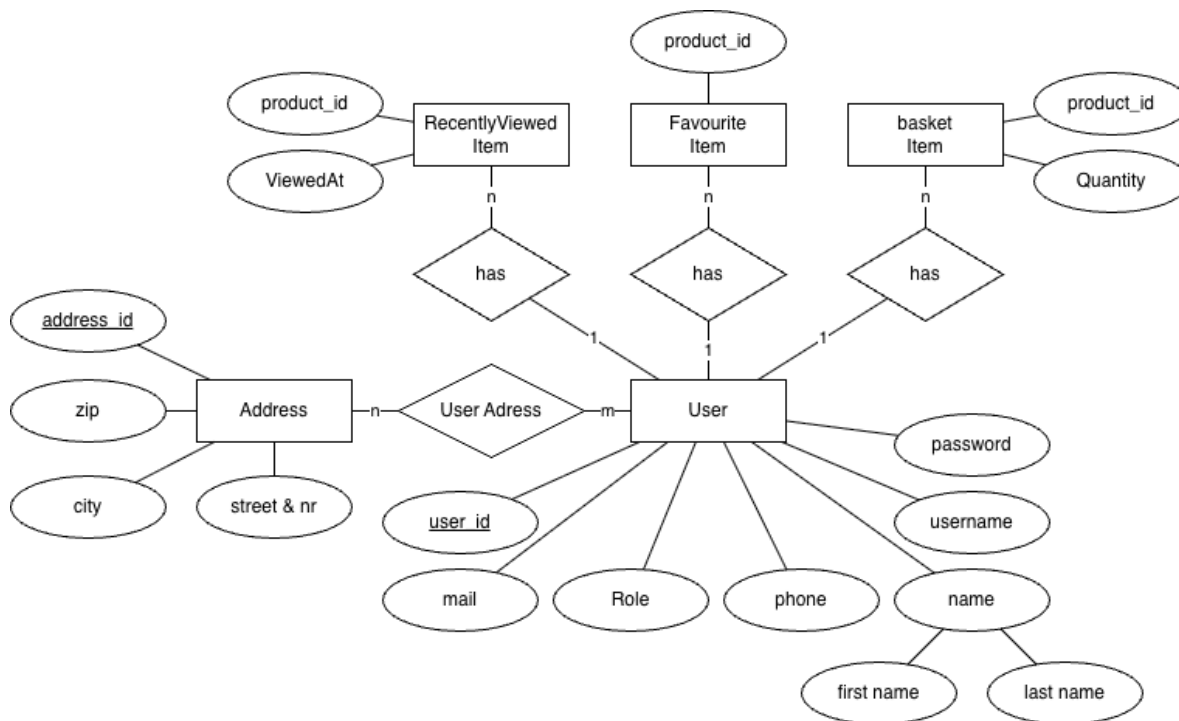


Figure 7: ER Diagram of the User Database

9.4 Styleguide

Introduction

This document is a Style Guide for the semester project. It covers the language C# and how to write code consistently. Example code and comments will be used to highlight the most important details. The Style Guide is based on Google's Style Guide for C# [4]. The code examples used are also based on Google's Style Guide [4], with modifications to better fit the group's use.

Naming Conventions

Summary

Code

- Classes, methods, enums, and namespaces are written in PascalCase.
- Variables and parameters are written in camelCase.
- Naming conventions are not affected by modifiers such as const, static, readonly, etc.
- Interfaces always start with "I".
- All code and comments in the code are written in English.

Files

- File names are in PascalCase.
- Where possible, the name of a file should match the name of the class it contains.

- It is preferred to have only one class per file.

Code Organisation

- “**using**” statements for namespaces and libraries are placed at the top of the file, before the file’s own namespace, in alphabetical order. The exception is System imports, which must always come first.
- The order of variable accessors in code should be: public, protected, private, virtual, static, readonly. Other accessors exist but are not relevant here.
- Order of elements within a class:
 - Enums
 - Static, const and Readonly
 - Fields and properties
 - Constructors
 - Methods

Other Important Rules

- Only one statement per line (a piece of code ending with “;”).
- Only one assignment per line.
- Indent with tabs, where one tab equals 4 spaces.
- When opening a code block, a new line must be added before the opening brace “{”.
- else and else if must be written on the same line as the closing brace “}” of the preceding code block.
- Braces must always be used, even when not required.
- A space must follow if/for/while keywords and commas.
- Spaces must be placed between all operators and values. This excludes shorthand operators (!, tilde, ++, -), which are written without spaces.
- When concatenating strings, either interpolation or the “+” operator may be used. Choose whichever is most readable.
- Avoid using “magic numbers”. Instead, declare variables with a named value first and then pass them to a method, or use named arguments.

9.5 Personas

This appendix contains common personas made across the team.

Customers

Søren

Role: Student/Private customer

Age: 23

Education: Studies software engineering at SDU

Location: Odense

History: Young student who needs help finding a new laptop for his studies. He is also a hobby-enthusiast who wants to upgrade his desktop PC with new components. He has a basic understanding of computers and is of course competent when it comes to software

Goal: Find a laptop for school and information on how to upgrade his desktop PC

Person

Quote: *"I'm not looking for someone to do the work for me, I just need the right tools and clear information to get it done right."*

Principles:

Søren would rather spend a long time learning how to do things himself rather than having to rely on others for help.

Primary Concerns & Pain Points

- He wants easy to understand guides on the best computers and components and how to upgrade his PC himself.
- He values an easy checkout process that is straightforward and manageable
- He wants to be able to easily compare information between products helping him choose the right one
- He is thorough when it comes to his purchases and therefore hates when information is hard to reach or not well displayed
- He gets frustrated when guides or information is poorly articulated or conveyed

Lars

Rolle: IT-ansvarlig i en mindre virksomhed

Beskrivelse: 45 år, 25 ansatte, står for indkøb af virksomhedens computere, skærme, osv. Har god teknisk forståelse, men har ikke tid til at bruge lang tid på research – foretrækker webshop hvor han hurtigt kan finde de rigtige produkter og bestille flere enheder på en gang.

Mål: At finde pålideligt IT-udstyr til virksomheden hurtigt og effektivt, så medarbejderne altid har det nødvendige udstyr.

Teknologi: Bruger webshop til at foretage online bestillinger.

Behov: Webshop, hvor han nem kan søge efter produkter, sammenligne specifikation af bestiller flere enheder samtidig. Også vigtigt at produktinformation og lagerstatus er korrekt.

Frustration: Hvis lagerstatus ikke passer, eller er uklart om et produkt er tilgængelig. Mister hurtigt tålmodigheden, hvis webshoppen er langsom eller gør det besværligt at bestille flere produkter på en gang.

Holger

Role: Loyal Local Customer/Tech Enthusias

Age: 55

Education: Senior Technician at a local manufacturing plant

Location: Living in the suburbs of Odense (Seden/Bullerup area)

History: Has been buying his PC parts and home appliances from Arnes Elektronik for 8 years.

Person

Quote: *"I'd rather pay 10 percent more for a machine that lasts and a person who knows how to fix it."*

Principles:

Holger is a "tinkerer." He's the guy neighbors call when their Wi-Fi goes down or their printer jams. He values Arnes Elektronik because he can talk to Arne about technical specs—something he can't do at ElgiFanten.

He is financially stable but, like Arne, he is pragmatic. He views electronics as investments, not disposable toys. He appreciates the "old school" service but is increasingly frustrated when he drives to the shop only to find out a part is backordered. He uses a smartphone for everything from banking to checking weather for his weekend sailing, so he expects a modern digital interface from his favorite local shop.

Primary Concerns & Pain Points

- **Wasted Trips:** Driving to the shop only to find the "Out of Stock" sign is his biggest frustration.
- **Lost Documentation:** He often loses physical warranty papers and prefers a digital "Customer Portal" where his receipts are stored.
- **Complex Checkout:** He hates long queues. If the shop's new system is slow or crashes while he's trying to pay, he loses confidence in the "expert" status of the store.
- **Privacy:** He is wary of big tech data mining. He trusts Arne with his data but wants to know it's stored securely and locally.
- **Inventory Transparency:** Wants to check online if a specific cable or component is in stock before driving to the shop.
- **Order History:** Needs to see his previous purchases (e.g., "Which ink cartridge did I buy last time?") without digging through paper receipts.
- **Reliable Notifications:** Expects an automated SMS or email the second his repair or special order is ready for pickup.
- **Seamless Pickup:** If he buys online, he wants a "Click and Collect" experience that is faster than standing in the main checkout line.

Workers

Lars

Role: Warehouse Manager (Mainly manages shipping and deliveries)

Age: 38

Education: Logistics

Location: Langeskov

History: Started working for Arne roughly 10 years ago and has since become good friends with Arne. Before this he was working as a warehouse assistant, not allowed to put his skillset to use. Working for Arne, he has been given free reins to optimize the warehouse and how orders are handled.

Goal: To efficiently track orders and have a clear overview of any new products added to the webshop. He also wants to avoid confused customer interaction that takes time away from his work.

Person

Quote: *"There's a specific kind of freedom that comes from knowing exactly where everything stands and leaving nothing left to chance."*

Principles:

Lars is a simple man that respects the flow. He values efficiency and takes pride when things run smooth and fast. He appreciates freedom in his work and dislikes noisy bosses and wants to be valued based on his work.

Primary Concerns & Pain Points

- **Accessible order status:** When a packaged is marked as “Shipped” in the Order Management System, it should be updated in the webshop for the user right away
- **Efficient warehouse updates:** When new deliveries arrive, should they quickly be added to the OMS and this in turn should automatically update stock status on the webshop
- **Confused customers:** He hates when customers call and ask because the webshop is not updating the order status fast enough
- **Inconsistencies between the Webshop:** He hates when there are clearly wares in stock, but the webshop says there are none left in stock, or the opposite and the store misses out on sales or they have to cancel orders and leave customers angry

Brian

Rolle: Lagermedarbejder.

Mål: Effektivitet og præcision i plukning af varer, pakning og afsendelse.

Teknologi: Bruger Orders Management System (OMS) til at se, så snart der kommer en ny ordre.

Behov: Data i realtid om ordrers produkter, ordredetaljer, og straks opdatering af lagerstatus. Så hans lager stemmer, og så kunderne ikke kan købe vare der ikke er på lager længere. En nem og overskuelig måde at se alle ordre og informationer. Evt. en måde at se lager lokation (f.eks. reol 120, hylde 2. Lokation.: 2120,). Når han markerer en pakke som ”Afsendt” i OMS, skal det slå igennem på webshoppen med det samme, så kunden selv kan se det på sin profil.

Frustration: Uoverensstemmelser mellem lagerstatus på webshop og det faktiske lager. Forsinkelser i systemet. Hvis han ikke får en indikation af, at der er kommet en ny ordre (notifikation, mail, etc). Kunder der ringer for at spørge om pakke er sendt, fordi webshoppen ikke opdaterer status hurtigt nok.

Karla

Role: Store manager and stakeholder

Age: 31

Education: Sales Assistant

Location: Reside in an apartment in Odense.

History: Started out as a helper, when Arne was the only one running the shop, but later got entrusted with managing the store, so Arne could focus on expanding the business and managing inventory.

Person

Quote: *”I need a system that is simple to engage with. It should automate the tedious tasks, while still keeping me in control.”*

Primary Concerns & Pain Points

- Assisting customers in finding the right products
- Documenting sales and orders
- Customer satisfaction
- Keeping tabs on how much is left in stock

Arne

Role: Founder & primary stakeholder

Age: 55

Education:

Location: Odense (Originally from Jylland)

History: Has a background in business and hobby in computers, turned his hobby into a business, fra Haderslev. Started his company in 2003 selling PCs and later branched out to selling components and providing information pages.

Goal: Expand his business with a complete and efficient but simple e-commerce solution

Person

Quote: *"I need a system that works as hard as I do, without costing a tractor's worth of licensing fees."*

Principles:

Arne is a business man at heart. He values quality over quantity and cherishes loyalty. In terms of skills he is more competent about hardware than software but keeps up to date in both the world of computers and software, as well as in how to optimize his business.

His Jutlandic roots make him naturally frugal; he hates waste and unnecessary subscriptions. While his business is growing, his time is increasingly tight, leaving him less time to visit his parents or pursue his hobbies. When he isn't at the shop, he enjoys playing Farming Simulator 16 (he insists the newer versions aren't as good!) and sponsoring local tractor pulling and mud racing events. He dreams of one day owning a tractor just like his father's.

Primary Concerns & Pain Points

- **Business operations:** Above everything he wants business operations to go smoothly without trouble. This for instance includes data accuracy and happy customers
- **Efficiency:** He also wants the store to run efficiently and is always looking to optimize this.
- **Loyalty:** He wants to keep a loyal customer base that always looks to his store first for help
- **Long term robustness:** He wants a webshop that not only is simple and functions smoothly, but also something timeless that won't need to be updated often.

9.6 Initial UML Class Diagram of Shop module

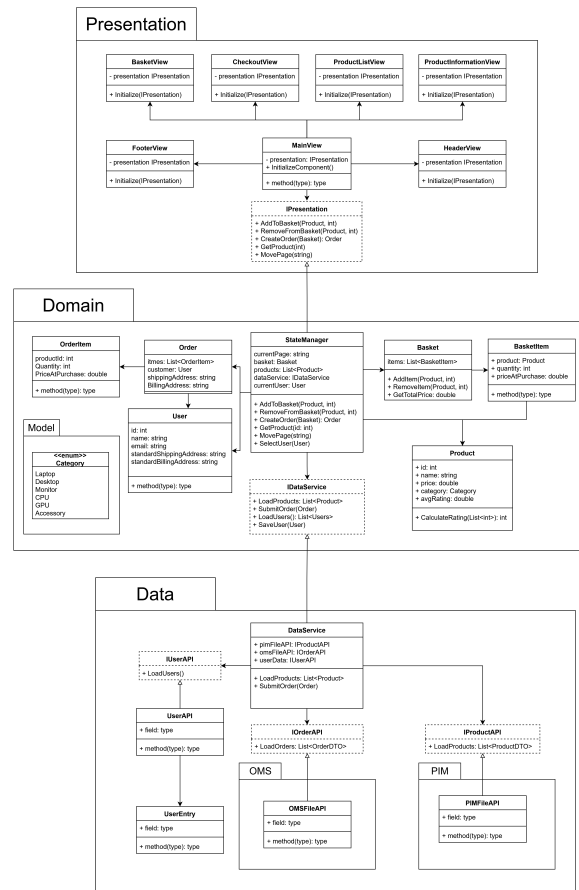


Figure 8: Initial UML Class Diagram of Shop module

9.7 Noun Verb Analysis

Noun-Verb Analysis

The shop exposes products and prices to customers through a graphical user interface (GUI).

Through the shop, customers (private and businesses) can search for- and buy products. Further, they can explore various content pages with helpful information.

The shop enables a customer to put products (along with a quantity) into a basket, show the total prices of items in the basket and allow for the customer to purchase these products.

The information shown in the shop is largely produced from information fetched from other systems. Products in the shop are fetched from the Product Information Management (PIM) system and once a user purchases the contents of his basket, an order is submitted to the OrderManagement System (OMS).

Content pages and products may be associated with media (such as images) that are fetched from the Digital Asset Management System (DAM).

Figure 9: Noun/verb analysis.

9.8 Supplementary Materials

The following project artefacts are included as supplementary materials in the attached zip file:

- **Sprint Reports** - Full sprint review and retrospective documents for all seven sprints.
- **Product Backlog** - Export of all backlog items created throughout the project, including user stories with title, assignee, priority, status and story points.
- **CRC Cards** - CRC cards produced during the initial analysis phase.
- **UML Diagrams** - Full-resolution versions of all diagrams referenced in the report.
- **Source Code** - Complete repository of the Shop module.